

Laporan Tugas Besar 1

IF2211 STRATEGI ALGORITMA

Pemanfaatan Algoritma Greedy dalam pembuatan bot permainan Battlecode 2025

Semester II Tahun 2025/2026



Kelompok 23 Stimathée Chalamet

Vincent Rionarlie	13524031
Emilio Justin	13524043
Philipp Hamara	13524101

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung



Daftar Isi

Daftar Gambar	iii
1 Deskripsi Tugas	1
1.1 Latar Belakang	1
1.2 Komponen Permainan	1
1.2.1 Objectives	1
1.2.2 Aturan Tiebreaker	1
1.2.3 Peta	1
1.2.4 Jarak Pandang dan Pergerakan	1
1.2.5 Tower dan Ruins	1
1.3 Sumber Daya	2
1.3.1 Cat (Paint)	2
1.3.2 Chips	2
1.4 Robot	2
1.4.1 Jenis Robot	3
1.5 Towers	3
1.5.1 Prosedur Pembangunan	3
1.5.2 Upgrade	3
1.6 Komunikasi	4
1.6.1 Pesan	4
1.6.2 Penanda	4
1.7 Actions dan Cooldown	4
1.7.1 Menyerang / Mewarnai	4
1.7.2 Bergerak	4
1.7.3 Menandai	4
1.7.4 Transfer Cat	4
1.7.5 Spawn Robot	4
1.7.6 Komunikasi	5
1.7.7 Self Destruct	5
2 Landasan Teori	6
2.1 Greedy	6
2.2 Cara Kerja Program	6
2.2.1 Arsitektur Program	6
2.2.2 Logika Bot	7
2.2.3 Menjalankan Bot	7
3 Aplikasi Strategi Greedy	8
3.1 Pemetaan	8
3.1.1 Tower	8
3.1.2 Bot	8
3.1.3 Communication	9
3.1.4 Pathfinding	9
3.1.5 Micro	9
3.1.6 Macro	9
3.2 Eksplorasi Heuristik	10
3.3 Analisis Efisiensi dan Efektivitas	12
3.4 The Chosen	13
4 Implementasi dan Pengujian	14
4.1 Himothee	14
4.1.1 Solusi Greedy yang Dipilih	14
4.1.2 Pseudocode Detail Solusi Greedy	14
4.1.3 Struktur Data yang Digunakan	16
4.1.4 Fungsi dan Prosedur Kunci	17
4.1.5 Intuisi Kenapa Greedy Ini Cocok	17
4.2 Thee	17



4.2.1	Solusi Greedy yang Dipilih	17
4.2.2	Pseudocode Detail	18
4.2.3	Struktur Data yang Digunakan	20
4.2.4	Fungsi dan Prosedur Kunci	21
4.2.5	Catatan Implementasi	21
4.3	Chamalet	21
4.3.1	Solusi Greedy yang Dipilih	21
4.3.2	Pseudocode Detail	21
4.3.3	Struktur Data yang Digunakan	27
4.3.4	Fungsi dan Prosedur Kunci	27
4.3.5	Catatan Inti Heuristik Chamalet	27
4.4	Hasil Pertandingan dan Analisis	28
4.4.1	Himothree vs Thee	28
4.4.2	Himothree vs Chamalet	29
4.4.3	Thee vs Chamalet	30
4.4.4	Local Postmortem Analysis	31
5	Kesimpulan dan Saran	32
5.1	Kesimpulan	32
5.2	Saran	32
	Lampiran	33
	References	34



Daftar Gambar

2.2.1 Tampilan Aplikasi Permainan	7
4.4.1 Himothee vs Thee	28
4.4.2 Himothee vs Chamalet	29
4.4.3 Thee vs Chamalet	30

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan.

Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, sehingga Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi *greedy* dalam membuat bot ini.

1.2 Komponen Permainan

1.2.1 Objectives

Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari **70%** petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Jika setelah **2000 ronde** tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir.

Penentuan pemenang dilakukan dengan urutan tiebreaker berikut:

1. Luas area yang sudah diwarnai di peta
2. Jumlah menara sekutu yang masih hidup
3. Total jumlah uang (*chips*)
4. Jumlah total cat (*paint*) yang dimiliki semua robot dan menara
5. Jumlah robot sekutu yang masih hidup
6. Jika masih seri, tim dipilih secara acak (*uniform random*)

1.2.3 Peta

Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara **20×20 hingga 60×60** petak (inklusif).

Titik kiri bawah peta memiliki koordinat (0,0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan dengan objek `MapLocation` yang menyimpan nilai x dan y .

Agar tidak ada peta yang menguntungkan salah satu pemain, dijamin bahwa peta bersifat simetris (baik melalui rotasi maupun refleksi).

1.2.4 Jarak Pandang dan Pergerakan

Robot selalu bisa melihat fitur peta dan robot lain dalam jarak hingga $\sqrt{20}$ (sekitar 4.47 petak).

Unit tidak bisa bergerak melewati menara atau robot lain. Peta mungkin memiliki tembok (*walls*), yang tidak bisa dilewati atau diwarnai. Tembok maksimal hanya **20%** dari seluruh peta.

Robot juga tidak bisa melewati atau mewarnai reruntuhan (*ruins*).

1.2.5 Tower dan Ruins

Menara juga berfungsi sebagai zona spawn tim dan bisa menyerang unit musuh.

Untuk membangun menara, tim harus mewarnai pola khusus di atas reruntuhan (*ruins*), yaitu zona 5×5 yang sudah ditentukan di peta.

Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah ada. Reruntuhan tidak bisa dipindah atau dihancurkan.

- Pusat reruntuhan berjarak minimal 5 petak satu sama lain

- Zona 5×5 di sekitar pusat reruntuhan dijamin bebas tembok
- Setiap tim memulai permainan dengan 2 menara:
 - 1 menara cat
 - 1 menara uang

Reruntuhan hanya bisa dilihat oleh robot atau menara jika berada dalam radius penglihatan.

1.3 Sumber Daya

1.3.1 Cat (Paint)

Cat adalah sumber daya utama permainan. Digunakan untuk mengklaim wilayah dan menghitung skor.

Setiap robot dan menara menyimpan cat secara individu, dengan kapasitas maksimal tergantung tipenya.

Robot yang cat-nya sedikit akan mengalami cooldown lebih lama atau bahkan tidak bisa beraksi.

Cat bisa:

- Diisi ulang dari *mopper*
- Ditarik dari menara penghasil cat

Aturan Penalti Cat

Jika cadangan cat hanya $X\%$ penuh dan $X < 50$ maka cooldown meningkat:

$$(100 - 2X)\%$$

Jika cadangan cat habis:

- Robot tidak bisa bergerak atau melakukan aksi
- Robot kehilangan 20 HP per ronde

Robot spawn dengan cadangan cat 100% penuh.

1.3.2 Chips

Chips digunakan untuk membangun unit dan menara.

- Chips dihasilkan setiap ronde oleh menara penghasil uang
- Dapat digunakan oleh semua unit sekutu
- Setiap tim memulai permainan dengan **2500 chips**

1.4 Robot

Semua robot dapat melakukan aksi seperti:

- Bergerak
- Merasakan lingkungan (*sensing*)
- Berkomunikasi

Permainan berjalan berbasis giliran (*turn-based*) dan dibagi menjadi ronde.

Setiap robot memiliki:

- HP (hitpoints)
- ID unik (tidak kurang dari 10000)
- Cadangan cat dengan kapasitas tertentu

Robot tidak dapat berbagi variabel statis karena setiap robot dijalankan pada JVM terpisah.



1.4.1 Jenis Robot

Soldier

- HP: 250
- Kapasitas cat: 200
- Biaya: 200 cat + 250 chips
- Serangan: satu petak (radius aksi 3)
- Damage menara musuh: 50
- Biaya serangan: 5 cat
- Cooldown aksi: 10

Splasher

- HP: 150
- Kapasitas cat: 300
- Biaya: 300 cat + 400 chips
- Serangan: area melingkar radius 2
- Damage menara musuh: 100
- Biaya serangan: 50 cat
- Cooldown aksi: 50

Mopper

- HP: 50
 - Kapasitas cat: 100
 - Biaya: 100 cat + 300 chips
- Kemampuan utama:
- Menghapus cat musuh
 - Mengambil cat dari robot musuh
 - Transfer cat ke sekutu
 - Serangan ayunan mop

1.5 Towers

Menara hanya dapat dibangun di atas reruntuhan.

1.5.1 Prosedur Pembangunan

1. Gunakan `markTowerPattern()` untuk menandai pola 5×5
2. Warnai pola sesuai penanda
3. Gunakan `completeTowerPattern()` untuk membangun menara

Biaya penandaan pola: **25 cat**.

1.5.2 Upgrade

Menara:

- Dibangun pada level 1
- Bisa di-upgrade menggunakan `upgradeTower()`

Batas maksimal menara per tim: **25 menara**.

1.6 Komunikasi

Robot tidak bisa berbagi variabel sehingga komunikasi sangat penting.

1.6.1 Pesan

Pesan memiliki batas:

- Robot: maksimal 1 pesan per ronde
- Menara: maksimal 20 pesan per ronde

Isi pesan:

- Integer 32-bit
- Nomor ronde
- ID pengirim

Pesan hanya disimpan selama **5 ronde**.

1.6.2 Penanda

Penanda dapat menunjukkan:

- Warna utama
- Warna sekunder

Penanda hanya terlihat oleh robot sekutu dan tidak memengaruhi kepemilikan wilayah.

1.7 Actions dan Cooldown

Robot dapat melakukan berbagai aksi:

1.7.1 Menyerang / Mewarnai

Dilakukan dengan fungsi:

`attack()`

Syarat:

- Action cooldown < 10

1.7.2 Bergerak

Dilakukan dengan:

`move()`

Syarat:

- Movement cooldown < 10
- Petak tujuan harus kosong

1.7.3 Menandai

Dilakukan dengan:

- `mark()`
- `removeMark()`

Biaya: **1 cat**

1.7.4 Transfer Cat

Dilakukan oleh **Mopper** menggunakan:

`transferPaint()`

1.7.5 Spawn Robot

Dilakukan oleh menara menggunakan:

`buildRobot()`



1.7.6 Komunikasi

- `sendMessage()`
- `readMessages()`

1.7.7 Self Destruct

Robot dapat menghancurkan diri sendiri menggunakan:
`disintegrate()`

Spesifikasi lengkap terkait permainan dapat diakses melalui pranala berikut <https://releases.battlecode.org/specs/battlecode25/3.1.0/specs.pdf>.

2 Landasan Teori

2.1 Greedy

Algoritma *greedy* adalah algoritma yang memecahkan persoalan secara langkah per langkah sedemikian sehingga pada setiap langkah mengambil pilihan yang terbaik yang dapat diperoleh pada saat itu tanpa memperhatikan konsekuensinya ke depan dan "berharap" bahwa dengan memiliki optimum lokal pada setiap langkah akan berakhir dengan optimum global. Terdapat sifat-sifat algoritma *greedy*, antara lain:

1. Greedy Choice Property: Solusi optimal dapat dibangun dengan memilih **keputusan lokal terbaik** terlebih dahulu.
2. Optimal Substructure: Solusi optimal dari persoalan dibentuk dari solusi optimal subpersoalannya.

Ide algoritma *greedy* dapat ditemukan melalui prosedur di bawah ini:

1. Rumuskan persoalan optimasi dengan pilihan greedy
2. Tunjukkan adanya optimal substructure
3. Buktikan (secara matematis) adanya solusi *greedy*-optimal

Komponen algoritma *greedy*:

1. Himpunan kandidat, C : berisi kandidat yang akan dipilih pada setiap langkah
2. Himpunan solusi, S : berisi kandidat yang sudah dipilih
3. Fungsi solusi: menentukan apakah solusi sudah lengkap
4. Fungsi seleksi: memilih kandidat berdasarkan strategi *greedy* tertentu. Strategi *greedy* ini bersifat heuristik
5. Fungsi kelayakan: memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam himpunan solusi
6. Fungsi objektif: memaksimumkan atau meminimumkan

Dengan menggunakan komponen di atas, maka dapat dikatakan bahwa algoritma *greedy* melibatkan pencarian sebuah himpunan bagian, S , dari himpunan kandidat, C ; yang dalam hal ini, S harus memenuhi beberapa kriteria yang ditentukan, yaitu S menyatakan suatu solusi dan S dioptimasi oleh fungsi objektif.

2.2 Cara Kerja Program

2.2.1 Arsitektur Program

Secara umum, proyek bot ini dikembangkan menggunakan bahasa pemrograman Java dengan sistem *build automation* **Gradle**. Gradle berperan penting dalam mengelola dependensi, mengkompilasi kode sumber, serta men-deploy dan menjalankan *game engine* yang menyimulasikan arena pertandingan.

Di dalam lingkungan simulasi tersebut, setiap bot memiliki satu *method entry point* utama, yaitu `run()` pada kelas `RobotPlayer` di dalam file `RobotPlayer.java`. File ini bertindak layaknya fungsi `main` untuk masing-masing bot. Saat sebuah bot di-*spawn* atau dibuat oleh *engine*, *method* ini hanya dipanggil satu kali, kemudian berjalan di dalam sebuah *infinite loop* (yang mana setiap iterasinya merepresentasikan satu giliran atau *turn* bagi bot).

Pada setiap giliran tersebut, bot mengeksekusi serangkaian aksinya dengan memanggil *method* logika spesifiknya, dan wajib diakhiri dengan pemanggilan `Clock.yield()` untuk menyerahkan kembali kendali komputasi kepada *engine*. Jika bot gagal memanggil `Clock.yield()` hingga batas komputasi lokalnya habis, gilirannya akan diakhiri secara paksa. Sebaliknya, jika *method* `run()` memanggil `return` (keluar dari *infinite loop*), bot tersebut dianggap mati secara sistem. Spesifikasi mendetail dari API yang disediakan dapat diakses melalui pranala berikut <https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/package-summary.html>.

2.2.2 Logika Bot

Seperti yang sudah disebutkan di arsitektur program, fungsi `main` bot ada di method `run()` yang menerima argumen sebuah objek `RobotController rc`. Secara sederhana, isi dari method tersebut adalah pemanggilan method logika masing-masing jenis bot. Logika *greedy* yang sudah didesain terhadap masing-masing bot akan dijalankan sehingga bot akan melakukan aksi sesuai dengan jenis bot tersebut. Pengimplementasian algoritma *greedy* untuk masing-masing bot pada dasarnya dapat dilakukan di method-method berikut:

1. `runTower()`
2. `runSoldier()`
3. `runMopper()`
4. `runSplasher()`

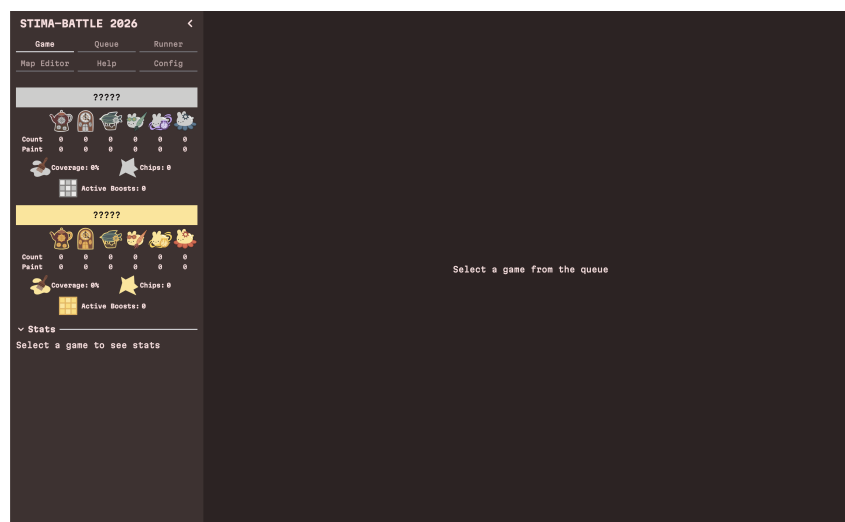
Meskipun demikian, kami merancang implementasi yang modular di dua bot sehingga lebih rapi dan lebih fokus untuk mengembangkan masing-masing logika botnya.

2.2.3 Menjalankan Bot

Aplikasi permainan yang sudah dibuka memiliki beberapa *interface* yang dapat digunakan untuk memulai pertandingan antara dua bot (lihat Gambar 2.2.1). Prosedur menjalankan bot untuk dipertandingkan:

1. Klik menu **Runner**
2. Pilih dua jenis bot yang merepresentasikan masing-masing tim (Tim A dan B)
3. Centang satu atau lebih peta yang ingin dipertandingkan
4. Klik tombol run

Simulasi pertandingan antara kedua bot yang sudah dipilih akan ditampilkan di panel sebelah kanan.



Gambar 2.2.1: Tampilan Aplikasi Permainan

3 Aplikasi Strategi Greedy

3.1 Pemetaan

Setelah mengamati, menonton *lecture* MIT, dan mengerti konsep keseluruhan permainan Battlecode 2025, terdapat beberapa komponen permainan esensial yang logikanya harus dikembangkan, antara lain:

1. Tower
2. Bot
3. Communication
4. Pathfinding
5. Micro
6. Macro

Oleh karena enam komponen tersebut memiliki kandidat strategi-strategi yang mungkin untuk didesain, pemetaan persoalan Battlecode 2025 menjadi elemen-elemen algoritma *greedy* akan dibagi menjadi enam subpersoalan.

3.1.1 Tower

Tower digunakan untuk membuat Bot baru, menyerang musuh yang ada di dekatnya, dan mencari *resources* sesuai dengan jenis Tovernya (kecuali Defense Tower). Berikut pemetaannya:

- Himpunan Kandidat : Himpunan aksi yang bisa dilakukan oleh Tower pada satu giliran,
- Himpunan Solusi : Keputusan akhir untuk melakukan *spawn* bot tertentu atau menyerang target tertentu di giliran itu.
- Fungsi Solusi : Memeriksa apakah aksi yang dilakukan valid dan paling efektif di giliran tersebut.
- Fungsi Seleksi : Memilih aksi krusial berdasarkan kondisi instan (pemilihan target, pemilihan urutan *spawn* Bot).
- Fungsi Kelayakan : Memeriksa apakah Tower memiliki cukup *resources* untuk melakukan *spawn*, atau apakah target serangan berada dalam jangkauan yang valid.
- Fungsi Objektif : Memaksimalkan pertahanan area dan efisiensi konversi *resources* menjadi Bot.

3.1.2 Bot

Bot adalah unit yang akan melakukan aksi untuk memenangkan permainan, terdapat tiga jenis Bot yang masing-masing melakukan aksi yang ide dasarnya sama. Berikut pemetaannya:

- Himpunan Kandidat : Himpunan semua kemungkinan aksi yang bisa dilakukan Bot berdasarkan observasi di sekitarnya pada giliran tersebut.
- Himpunan Solusi : Aksi atau kombinasi aksi terpilih (bergerak atau kemampuan internal Bot) yang akhirnya dieksekusi oleh Bot pada satu giliran.
- Fungsi Solusi : Memeriksa apakah Bot sudah mengeksekusi aksi paling optimal yang paling mungkin di giliran itu.
- Fungsi Seleksi : Memilih kandidat aksi yang memberikan keuntungan lokal terbesar.
- Fungsi Kelayakan : Memeriksa apakah aksi yang dikandidatkan tersebut valid dan legal sesuai dengan aturan permainan.
- Fungsi Objektif : Mencari satu aksi valid yang paling menguntungkan tepat di giliran tersebut.

3.1.3 Communication

Agar dapat mengetahui apa yang sedang terjadi *real time* di permainan, Bot dan Tower dapat saling bertukar pesan untuk mengabarkan kejadian yang sudah terjadi. Berikut pemetaannya:

- Himpunan Kandidat : Kumpulan informasi lokal yang diamati oleh Bot dan berpotensi untuk dikirimkan.
- Himpunan Solusi : Pesan spesifik yang dikodekan dan dikirim pada giliran tersebut.
- Fungsi Solusi : Memeriksa apakah pesan berhasil dikirimkan.
- Fungsi Seleksi : Memprioritaskan pengiriman informasi yang memiliki urgensi paling tinggi bagi *ally*.
- Fungsi Kelayakan : Memeriksa apakah batas pengiriman pesan pada giliran tersebut masih tersedia.
- Fungsi Objektif : Memaksimalkan pengetahuan global tim.

3.1.4 Pathfinding

Pathfinding memungkinkan Bot untuk melakukan penelusuran peta dan pergerakan yang baik di peta, dibutuhkan Pathfinding yang optimal agar semua Bot bergerak ke arah yang paling optimal dan efisien. Berikut pemetaannya:

- Himpunan Kandidat : Daftar petak yang berdekatan atau arah pergerakan yang memungkinkan.
- Himpunan Solusi : Satu arah pergerakan yang dipilih untuk dieksekusi oleh Bot.
- Fungsi Solusi : Memastikan Bot berpindah setidaknya satu langkah lebih dekat ke tujuan jika memungkinkan.
- Fungsi Seleksi : Mengevaluasi setiap arah dan memilih petak yang paling optimal di giliran tersebut menuju suatu objektif yang ingin dicapai di saat itu.
- Fungsi Kelayakan : Memeriksa apakah gerakan dapat dilakukan.
- Fungsi Objektif : Meminimalkan jumlah giliran yang dibutuhkan untuk mencapai tujuan dengan rute paling efisien.

3.1.5 Micro

Micro pada Battlecode 2025 lebih mengacu pada aksi pertempuran yang dilakukan masing-masing Bot kepada musuh. Berikut pemetaannya:

- Himpunan Kandidat : Daftar aksi mikro yang tersedia ketika bertemu dengan musuh.
- Himpunan Solusi : Urutan aksi taktis yang dieksekusi di tengah pertempuran.
- Fungsi Solusi : Memastikan giliran berakhir dengan posisi atau status unit yang lebih menguntungkan daripada sebelumnya.
- Fungsi Seleksi : Memilih aksi yang memaksimalkan pengaruh buruk kepada musuh sambil meminimalkan pengaruh buruk kepada teman.
- Fungsi Kelayakan : Memastikan komputasi tidak melebihi *bytecode*.
- Fungsi Objektif : Memaksimalkan aksi spesifik yang merugikan musuh dan meminimalkan kerugian teman.

3.1.6 Macro

Macro merujuk pada *decision making* selama permainan berlangsung. Macro dapat menentukan tujuan yang ingin dicapai di fase tertentu. Berikut pemetaannya:

- Himpunan Kandidat : Daftar tujuan strategis global yang ingin dicapai.
- Himpunan Solusi : Arahan strategi global yang sedang dijalankan oleh keseluruhan Unit.
- Fungsi Solusi : Arahan strategi berhasil didistribusikan secara global (biasanya lewat Communication).

- Fungsi Seleksi : Mengevaluasi kondisi global saat ini dan memilih strategi yang memberikan probabilitas kemenangan tertinggi saat itu juga.
- Fungsi Kelayakan : Memeriksa apakah semua Unit memiliki *resources* atau kondisi yang diinginkan memungkinkan untuk mendukung strategi global tersebut.
- Fungsi Objektif : Memaksimalkan penguasaan peta dan meraih kemenangan.

3.2 Eksplorasi Heuristik

Pemetaan yang sudah dilakukan di atas membuka banyak kemungkinan strategi solusi yang mungkin untuk diterapkan. Terdapat tiga Unit yang kami rancang dengan nama masing masing, **Himothée**, **Chamalet**, dan **Thee**. Di bawah ini adalah kandidat solusi berlandaskan *greedy* yang kami rancang terhadap masing-masing komponen esensial permainan pada masing-masing Unit, dengan komponen Bot dibagi menjadi tiga, yaitu Soldier, Mopper, dan Splasher:

Communication

- **Himothée** : Mengirim status kesimetrian peta (`SYM_UPDATE`) guna memproyeksikan koordinat Tower musuh. Diperkuat dengan komunikasi dinamis `SAVE_CHIPS` ketika terdeteksi reruntuhan di sekitar menara, serta pembagian koordinat Tower lawan yang dijumpai (`ENEMY_TOWER`) ke seluruh tim tanpa harus semuanya bereksplorasi.
- **Chamalet** : Menggunakan sistem komunikasi satu arah yang krusial dari Soldier ke Tower terdekat (`MSG_SAVE_CHIPS`). Ketika Soldier menemukan *ruin* yang dapat dibangun, pesan dikirim agar Tower menghentikan produksi unit sementara waktu (*chip buffering*) guna memastikan pembangunan Tower baru dapat segera dieksekusi.
- **Thee** : Mengandalkan Mopper sebagai utusan (*Messenger*) yang berpatroli dan secara proaktif mengirim sinyal penahanan *Chips* (`SAVING`) langsung ke kumpulan menara sekutu. Secara periodik menyebarkan akumulasi jumlah musuh yang terdeteksi ke jaringan sekutu.

Pathfinding

- **Himothée** : Memadukan algoritma **bug0** untuk peluncuran garis lurus efisien, dan beralih dinamis ke **bug2** saat menghadapi dinding padat. Proses pergerakan dikalkulasi oleh *moveTile-Score* agar unit tetap mempertahankan kontrol zona sembari menjajah menjauhi pusat sekutu dengan membiaskan arah rotasi eksplorasi menuju pojok peta terluar (*corners*).
- **Chamalet** : Mengimplementasikan algoritma **bug2** terarah yang diperkuat dengan *tile-scoring* saat bot bergerak bebas. Saat mengeksplorasi, ia secara *greedy* menyebar menjauhi sentroid kepadatan kawanannya.
- **Thee** : Mengimplementasikan algoritma **bug2** dengan memori *state* presisi mutlak. Setiap kali target berubah, bot langsung meluncur lurus (*greedy euclidian*) mengabaikan sisa *obstacle tracing* lama demi mencapai target terbaru.

Micro

- **Himothée** : Menerapkan threshold mundur (*retreat*) anti-sinkron secara heuristik per bot bergantung pada ID agar unit tidak menumpuk saat kembali mundur. Secara ofensif, status *retreat* dapat dibatalkan jika musuh muncul secara mendadak demi mempertahankan garis depan lewat pertarungan (*lifesteal*).
- **Chamalet** : Berfokus pada *kiting* agresif terhadap Tower lawan untuk meminimalkan kehilangan HP. Bot secara *greedy* membidik unit dengan HP paling sedikit untuk mempercepat defisit jumlah unit lawan.
- **Thee** : Secara *greedy* menggunakan sistem cat `PaintType.ALLY_SECONDARY` untuk menunjuk dan mengunci secara spasial titik kotor, memaksa kelas Mopper wajib datang.

Macro

- **Himothée** : Strategi rotasi *scale-dependent*. Pada awal ($round < 50 \times scale$) memproduksi *Soldiers* dan *Moppers*. Pertengahan ($< 100 \times scale$) membuka peluang lahirnya *Splasher*. Masuk ronde akhir ($> 200 \times scale$), unit disetel dalam state `ATTACKING` invasif. Prioritas utama ada pada pelebaran pembangunan Tower (*Expand* $>$ *Upgrade*) karena harganya mutlak lebih murah. *Upgrade* hanya akan dieksekusi bila formasi telah matang ($Tower \geq 4$ dan $Chips \geq 3500$) melampaui tabungan formasi awal.

- **Chamalet** : Membagi rotasi *spawning*. Awal rotasi untuk eksplorasi kilat (S-S-M), namun di ronde menengah ke atas digeser untuk ekspansi dan pembersihan (S-S-Sp-Sp-M-M-M), sambil heuristik memelihara rasio *Paint Tower*.
- **Thee** : Pemilihan antrean *spawning* dipadukan oleh *multiplier* area peta. Produksi dikendalikan oleh hierarki pelit (*skip turns*): menara berhenti memproduksi bot secara serentak (70 *turn*) ketika komunikasi diterima.

Soldier

- **Himothée** : Unit dipisah menjadi dua peranan krusial berdasarkan *modulo* unit ID:
 1. **SRP Specialist** (~ 33% jumlah): Secara mutlak hanya mencari kandidat titik sentral terdekat dan membangun utilitas *Special Resource Pattern* (SRP) berskala 5×5 , utamanya diprioritaskan bersinggungan dengan bangunan *tower* sekutu.
 2. **Regular Explorer**: Melakukan inisiasi bangunan *tower pattern*, mengirim sinyal ketersediaan reruntuhan agar *tower* fokus menabung dana spesifik sejumlah estimasi *Chips* bangunan.
- **Chamalet** :
 1. Mengisi ulang amunisi cat saat indikator menipis.
 2. Bergerak mengklaim reruntuhan dari wilayah tak bertuan.
 3. Melakukan penyerangan *micro* proaktif melawan Tower musuh.
 4. Menjelajah dan menandai teritori acak yang dilewati.
- **Thee** :
 1. Pergi memprioritaskan reruntuhan yang masih kosong terdekat terlebih dahulu.
 2. Melakukan penandaan pelimpahan tugas kepada Mopper pada reruntuhan yang tertutup noda cat musuh.
 3. Menerapkan toleransi yang sangat ketat dalam pengisian ulang amunisi cat ke menara.

Mopper

- **Himothée** :
 1. Secara instan memprioritaskan menebas bila ada ≥ 2 musuh dalam jangkauan *swing*, atau menyerang satuan robot berelemen cat terbesar guna memanen suplai cat murni secara instan tanpa perlu pulang (meskipun amunisi awal tinggal seujung kuku).
 2. Sukarela mentransfer tabungan *paint* saat cadangan rasio berada di atas 40% untuk unit-unit *ally* di garis depan yang nyawanya tinggal setengah (50%).
 3. Mendeteksi dan langsung mengubah *state*-nya menjadi penjaga konstruksi ketika mendeteksi reruntuhan sekutu tertutup cat musuh.
- **Chamalet** :
 1. Memfokuskan diri sebagai spesialis pelindung reruntuhan.
 2. Mengeksekusi prioritas ayunan pertarungan *Swing* bila menguntungkan.
 3. Bertindak sebagai pendonor tabungan *Paint* cadangan kepada sekutu yang membutuhkan.
- **Thee** :
 1. Berperan penting sebagai kurir *skip turns* ke markas dengan menabrak batasan eksplorasi.
 2. Prioritas pergerakan utama dialihkan pada titik *Marking* unit kawan.
 3. Berotasi membatasi bersih spot target dan menghilangkan *Marking* jika sudah aman.

Splasher

- **Himothée** : Mengagresi teritorial musuh dengan penetrasi dalam secara cerdas:
 1. Berlari condong secara spesifik memprioritaskan Tower lawan berdasarkan *state broadcast* kawan, memori Tower yang telah tumbang, maupun memproyeksikan asumsi kesimetrian posisi sudut formasi kawan.
 2. Hanya menghamburkan amunisi luas saat ubin *enemy* menumpuk di atas 8 *tiles* area, melindungi wilayah struktur utilitas kawan sepenuhnya secara statis, sembari menjaga jarak mundur (*Kite*) sekitar radius 16 unit proyektil dari ancaman tower musuh.
- **Chamalet** :

1. Kalkulasi penentuan area tembak yang **mutlak di-blacklist** dari perbatasan wilayah *ruin* sekutu.
 2. Bermanuver proaktif untuk melahirkan wilayah dominasi peta paling bernilai tinggi (*scoring*).
- **Thee :**
 1. Beroperasi stabil dan adaptif terhadap instruksi *spawning* rotasi makro.
 2. Bergerak dan mengeksekusi serangan AOE di luar pengawasan menara.

Tower

- **Himothee :**
 1. Menerima *symmetric messages*, rekam jejak wilayah musuh, dan melarang *Upgrade Level* secara ketat demi membuka modal tabungan yang diarahkan sepenuhnya dalam ekspansi kloning infrastruktur yang jauh lebih murah (1000 *Chips*).
 2. Mengunci jumlah simpanan secara kalkulatif ($1000 \times$ jumlah reruntuhan tak bertuan di sekitar). Operasi penghentian produksi ini secara instan dilepas jika *limit round* telah kedaluwarsa.
 3. Sengaja melahirkan unit pertempuran di spot tak berpemilik atau bernoda musuh agar unit baru yang melangkah langsung otomatis menginisiasi petak tersebut menjadi kontrol teritori.
- **Chamalet :**
 1. Berperan murni sebagai koordinator penimbun dana pasif begitu pesan prioritas *SAVE* diterima.
 2. Menembak serentak basis pendaratan target musuh terjangkau yang nyawanya menduduki angka paling kecil.
- **Thee :**
 1. Meringankan pengeluaran awal permainan agar konstan bersisa pada margin setidaknya 1000 poin ketersediaan *Chips*.
 2. Sengaja memogokkan sistem antrean produksinya sama sekali menurut Mopper.

3.3 Analisis Efisiensi dan Efektivitas

Analisis di bawah ini merupakan sintesis dari kedelapan komponen esensial permainan yang terintegrasi di dalam sistem bot.

Chamalet

- **Efisiensi**

Secara komputasi (*bytecode*), bot ini tergolong ringan dan efisien. Kalkulasi pergerakan murni bertumpu pada *tile-scoring* (+5, +2, -6) yang konstan dan evaluasi pencarian musuh bernyawa terendah (*Lowest HP*). Namun, efisiensi waktu dan daya (*in-game resources*) kurang maksimal; agresivitas *greedy* dengan ambang batas *refuel* yang ketat membuat unit membuang banyak *turn* hanya untuk bolak-balik mengisi cat.
- **Efektivitas**

Sangat tangguh dalam pertempuran jarak dekat skala kecil (*micro-skirmish*). Mekanik *kiting* proaktif dari Soldier, kalkulasi *Mop Swing* (≥ 2 penetrasi musuh), serta algoritma ketat *Splasher* yang melarang proyektil merusak *pattern* bangunan kawan (*friendly-fire blacklist*) menjamin formasi tetap solid untuk menguasai arena di awal pertarungan.

Thee

- **Efisiensi**

Sangat unggul dalam efisiensi komputasi *state machine* berkat arsitektur pendelegasian tugas. Mekanik Soldier yang sebatas mengidentifikasi, memberi *mark* khusus (*ALLY_SECONDARY*), lalu pergi secara absolut meniadakan isu *overlapping action*. Integrasi penahanan modal statis (Tower mogok produksi selama 70 *turns* via *Messenger*) menekan buangan *chips* berlebih pada unit yang tidak krusial.

- **Efektivitas**

Unggul dalam kestabilan ekspansi infrastruktur. Algoritma **bug2** bot ini memiliki memori yang otomatis membuang transisi hambatan *tracing* jika target direset, yang memaksa presisi *Euclidean* sempurna. Efektivitasnya sangat tinggi untuk makro teritorial, meski kerentanan ada pada besarnya tumpuan peran Mopper sebagai satu-satunya *Messenger* tim.

Himothée

- **Efisiensi**

Bot ini mengonsumsi *bytecode* yang sangat berat akibat kalkulasi asimilasi pola (pemindaian *matrix* titik SRP 5×5), pelacakan simetri peta, hingga manipulasi rekaman letak Tower. Namun secara paradoks, bot ini meraih efisiensi *in-game resources* paling sempurna. *Threshold retreat* dinamis berbasis sisa pembagian unit ID mengatasi kemacetan massal (*traffic logic*). Mopper dirancang *self-sustain* memanfaatkan cairan pampasan musuh (*lifesteal*) tanpa perlu pulang, ditambah pendedikasian $\approx 33\%$ Soldier eksklusif sebagai SRP Specialist penyuplai ekonomi *late-game*.

- **Efektivitas**

Efektivitas penyerangan (*deep push*) sangat buas dan mematikan. Penggabungan transisi cepat *bug0* ke *bug2*, memori Splasher pada Tower yang sudah hancur, hingga kemampuan memproyeksikan koordinat jantung pertahanan musuh lewat deduksi simetris spasial memastikan invasi tim langsung menghunus basis lawan tanpa membuang *turn* eksplorasi meraba-raba.

3.4 The Chosen

Perumusan ide strategi solusi permainan Battlecode 2025 ini beranjak dari ide dasar yang mirip dengan bot **Chamalet**, kemudian diperluas menjadi bot **Thee** dan **Himothée**. Kami fokus mengembangkan bot **Himothée** dengan mengintegrasikan ide heuristik yang bagus dan mekanisme adaptivitas yang baik. Pendekatan perpaduan *greedy* pada **Himothée** memungkinkan bot untuk tidak sekadar bereaksi terhadap kondisi lingkungan, melainkan aktif melacak tempo permainan sejak awal permainan.

Ide untuk mendeteksi kesimetrian peta secara dinamis, skalabilitas makro yang menyesuaikan dengan ukuran peta hingga spesialisasi peran pada masing-masing bot terbukti memberikan keunggulan ketika pertandingan memasuki fase *late-game*.

Pada akhirnya, **Himothée** versi 2.0 ditetapkan sebagai Bot utama kami.

4 Implementasi dan Pengujian

4.1 Himothee

4.1.1 Solusi Greedy yang Dipilih

Solusi yang digunakan pada Bot Himothee adalah **Hierarchical State-Based Greedy**. Setiap unit memilih *state* prioritas tertinggi terlebih dahulu (mis. RETREAT, ATTACKING, PAINTING_PATTERN), lalu di dalam *state* tersebut memilih aksi lokal terbaik berdasarkan skor heuristik saat ini.

4.1.2 Pseudocode Detail Solusi Greedy

Algorithm 1 Main Loop Per Unit (Greedy)

```
1: procedure RUNUNITTURN(rc)
2:   UPDATEFRIENDLYTOWERS(rc)
3:   UPDATESYMMETRYGUESS(rc)
4:   REPORTENEMYTOWERS(rc)
5:   if NEEDRETREAT(rc) then
6:     RUNRETREAT(rc)
7:     return
8:   end if
9:   if rc.type = SOLDIER then
10:    RUNSOLDIERGREEDY(rc)
11:   else if rc.type = MOPPER then
12:    RUNMOPPERGREEDY(rc)
13:   else if rc.type = SPLASHER then
14:    RUNSPLASHERGREEDY(rc)
15:   else
16:    RUNTOWERGREEDY(rc)
17:   end if
18: end procedure
```

Algorithm 2 Greedy Movement Selector (Bug0-style)

```
1: function GREEDYMOVE(current, target)
2:    $d \leftarrow \text{DIRECTIONTO}(\text{current}, \text{target})$ 
3:    $C \leftarrow [d, \text{left}(d), \text{right}(d), \text{left2}(d), \text{right2}(d)]$ 
4:    $\text{bestDir} \leftarrow \emptyset$ ,  $\text{bestScore} \leftarrow -\infty$ 
5:   for all  $x \in C$  do
6:     if CANMOVE( $x$ ) then
7:        $\text{next} \leftarrow \text{current} + x$ 
8:        $\text{score} \leftarrow \text{MOVETILESCORE}(\text{next})$ 
9:       if  $x = d$  then  $\text{score} \leftarrow \text{score} + 3$ 
10:      end if
11:      if  $x = \text{left}(d)$  atau  $x = \text{right}(d)$  then  $\text{score} \leftarrow \text{score} + 1$ 
12:      end if
13:      if  $\text{score} > \text{bestScore}$  then
14:         $\text{bestScore} \leftarrow \text{score}$ ,  $\text{bestDir} \leftarrow x$ 
15:      end if
16:    end if
17:  end for
18:  if  $\text{bestDir} \neq \emptyset$  then
19:    MOVE( $\text{bestDir}$ )
20:  else
21:     $\triangleright$  Fallback jika semua kandidat buruk/terblokir
22:    BUG2TRACE(target)
23:  end if
24: end function
```

Algorithm 3 Soldier Greedy: Build/Explore/Attack

```
1: procedure RUNSOLDIERGREEDY(rc)
2:   if state = EXPLORING then
3:     ruin  $\leftarrow$  FINDNEARESTUNCLAIMEDRUIN(rc)
4:     if ruin  $\neq \emptyset$  then
5:       state  $\leftarrow$  PAINTING_PATTERN
6:       paintingRuin  $\leftarrow$  ruin
7:       towerType  $\leftarrow$  DETECTORCHOOSETOWERTYPE(rc, ruin)
8:     else
9:       GREEDYMOVE(rc.loc, exploreTarget)
10:    end if
11:  else if state = PAINTING_PATTERN then
12:    tileAtk  $\leftarrow$  BESTATTACKABLEPATTERN TILE(rc, paintingRuin, towerType)
13:    tileMove  $\leftarrow$  NEARESTUNPAINTEDPATTERN TILE(rc, paintingRuin, towerType)
14:    if tileAtk  $\neq \emptyset$  then
15:      ATTACKPATTERN TILE(rc, tileAtk, towerType)
16:       $\triangleright$  Warna primer/sekunder mengikuti pola tower
17:    end if
18:    if CANCOMPLETE TOWERPATTERN(rc, paintingRuin, towerType) then
19:      COMPLETE TOWERPATTERN(rc, paintingRuin, towerType)
20:      state  $\leftarrow$  EXPLORING
21:      return
22:    end if
23:    if MOVEMENTREADY(rc) dan tileMove  $\neq \emptyset$  then
24:      MOVETOWARDPATTERN TILEWITHBOUND(rc, tileMove, paintingRuin)
25:    end if
26:  else
27:    target  $\leftarrow$  SELECTENEMYTARGETGREEDY(rc)
28:    if target  $\neq \emptyset$  then
29:      MOVEWITHBUG2FALLBACK(rc, target)
30:      if CANATTACK(rc, target) then ATTACK(rc, target)
31:      end if
32:    else
33:      state  $\leftarrow$  EXPLORING
34:    end if
35:  end if
36: end procedure
```

Algorithm 4 Mopper Greedy: Combat First

```
1: procedure RUNMOPPERGREEDY(rc)
2:   enemies  $\leftarrow$  SENSEENEMIES(rc)
3:   if LOWPAINT(rc) dan enemies =  $\emptyset$  then
4:     RUNRETREAT(rc); return
5:   end if
6:   if enemies  $\neq \emptyset$  then
7:     swing  $\leftarrow$  BESTSWINGDIRECTIONBYHITS(rc, enemies)
8:     if HITS(swing)  $\geq 2$  then
9:       MOPSWING(rc, swing)
10:    else
11:      t  $\leftarrow$  BESTENEMYBYPAINTEAMOUNT(rc, enemies)
12:      if t  $\neq \emptyset$  then ATTACK(rc, t)
13:      end if
14:    end if
15:  end if
16:  if ACTIONREADY(rc) then
17:    p  $\leftarrow$  BESTENEMYPAINTTILEINRANGE(rc)
18:    if p  $\neq \emptyset$  then ATTACK(rc, p)
19:    end if
20:  end if
21:  if ACTIONREADY(rc) then
22:    ally  $\leftarrow$  LOWESTPAINTRATIOALLYNEARBY(rc)
23:    if ally  $\neq \emptyset$  dan HASSPAREPAINT(rc) then
24:      TRANSFERPAINT(rc, ally)
25:    end if
26:  end if
27:  target  $\leftarrow$  SELECTMOPPERMOVETARGET(rc)
28:  if target  $\neq \emptyset$  dan MOVEMENTREADY(rc) then
29:    MOVEGREEDYSCORED(rc, target)
30:  end if
31: end procedure
```

Algorithm 5 Splasher Greedy: Splash Value Maximization

```
1: procedure RUNSPLASHERGREEDY(rc)
2:   if LOWPAINT(rc) then
3:     RUNRETREAT(rc); return
4:   end if
5:   push  $\leftarrow$  PICKPUSHTARGETGREEDY(rc)
6:   if ACTIONREADY(rc) then
7:     best  $\leftarrow$  FINDBESTSPLASHTILE(rc)
8:     if best  $\neq \emptyset$  then ATTACK(rc, best)
9:     end if
10:  end if
11:  if ENEMYTOWERTOOCLOSE(rc) then
12:    KITEAWAYFROMTOWER(rc)
13:  else
14:    MOVEGREEDYSCORED(rc, push)
15:  end if
16: end procedure
```

4.1.3 Struktur Data yang Digunakan

1. **knownTowers** (list lokasi tower ally)
Digunakan untuk retreat, referensi simetri, dan pengiriman pesan.
2. **knownEnemyTowers** (list lokasi tower musuh)
Menjadi target prioritas untuk mode menyerang.

3. **knownMoneyTowers** (set lokasi tower ekonomi)

Digunakan untuk estimasi pemasukan dan keputusan saving chips.

4. **state, preRetreatState**

Menyimpan state unit aktif dan state sebelum retreat untuk kembali ke mode semula.

5. **targetEnemyRuin, exploreTarget, pushTarget**

Pointer target aktif agar keputusan tidak berubah-ubah setiap turn.

6. **symHorizontal, symVertical, symRotational**

Menyimpan hipotesis simetri peta untuk proyeksi posisi lawan.

7. **paintTowerPattern, moneyTowerPattern**

Template pola primer/sekunder untuk validasi pengecatan saat membangun tower.

8. **Variabel pathfinding (isTracing, tracingDir, prevDest, dst.)**

Menyimpan konteks tracing agar fallback obstacle handling stabil.

4.1.4 Fungsi dan Prosedur Kunci

1. **UpdateFriendlyTowers**: memperbarui daftar tower ally terdeteksi.
2. **UpdateSymmetryGuess**: memvalidasi/eliminasi asumsi simetri dari observasi lokal.
3. **ReportEnemyTowers**: melaporkan tower musuh ke tim melalui messaging.
4. **NeedRetreat, RunRetreat**: kontrol survivability berbasis threshold paint.
5. **MoveTileScore, MoveGreedyScored**: inti pemilihan langkah greedy.
6. **MoveWithBug2Fallback**: fallback ketika jalur lurus gagal.
7. **BestAttackablePatternTile, NearestUnpaintedPatternTile**: percepat penyelesaian pola tower.
8. **FindBestSplashTile**: memilih pusat splash dengan nilai area maksimum.
9. **ChooseSpawnTypeByGamePhase, BestSpawnTileGreedy**: keputusan makro produksi unit oleh tower.

4.1.5 Intuisi Kenapa Greedy Ini Cocok

- Kompleksitas per turn rendah, sesuai batas komputasi pertandingan.
- Responsif terhadap perubahan lokal yang cepat.
- Tetap terkoordinasi karena berbagi informasi global dan pesan.
- Robust di map berobstacle karena ada fallback tracing.

4.2 Thee

4.2.1 Solusi Greedy yang Dipilih

Bot Thee menggunakan pendekatan **stateful greedy**, yaitu:

1. Prioritaskan kebutuhan paling mendesak terlebih dahulu (refill paint, cleaning mark, saving chips).
2. Pilih target lokal terbaik/terdekat pada giliran saat ini.
3. Eksekusi aksi langsung (move, attack, paint, spawn).
4. Gunakan bug2 saat jalur lurus ke target terhalang.

4.2.2 Pseudocode Detail

Algorithm 6 Main Loop Thee

```
1: procedure RUNUNITTURN( $rc$ )
2:   if  $rc.type = SOLDIER$  then
3:     RUNSOLDIERGREEDY( $rc$ )
4:   else if  $rc.type = MOPPER$  then
5:     RUNMOPPERGREEDY( $rc$ )
6:   else
7:     RUNTOWERGREEDY( $rc$ )
8:   end if
9: end procedure
```

Algorithm 7 Soldier Greedy

```
1: procedure RUNSOLDIERGREEDY( $rc$ )
2:   UPDATECLOSESTREFILL( $rc$ )
3:   if  $isRefilling$  and  $paint \geq PAINT\_HIGH$  then
4:      $isRefilling \leftarrow false$ 
5:   end if
6:   if  $\neg isRefilling$  and  $paint \leq PAINT\_LOW$  then
7:      $isRefilling \leftarrow true$ 
8:   end if
9:   if  $isRefilling$  then
10:    MOVETOREFILLANDTRANSFER( $rc$ )
11:    return
12:  end if
13:  if  $ruinRetargetCooldown > 0$  then
14:     $ruinRetargetCooldown \leftarrow ruinRetargetCooldown - 1$ 
15:    RUNEXPLORETURN( $rc$ )
16:    return
17:  end if
18:   $ruin \leftarrow FINDCLOSESTEMPTYRUINNOTOCCUPIED(rc)$ 
19:  if  $ruin \neq \emptyset$  then
20:    if  $ANYPATTERNCOMPLETE(rc, ruin)$  and  $\neg CANAFFORDCOMPLETION(rc)$  then
21:       $\triangleright$  Hindari loop ke ruin yang complete tapi chip belum cukup
22:      ABANDONRUINANDEXPLORE( $rc, ruin$ )
23:      return
24:    end if
25:    HANDLERUINBUILDING( $rc, ruin$ )
26:    return
27:  end if
28:  RUNEXPLORETURN( $rc$ )
29: end procedure
```

Algorithm 8 Handle Reruntuhan (Greedy Building)

```
1: procedure HANDLERUINBUILDING( $rc, ruin$ )
2:   Bergerak ke posisi efektif di sekitar ruin (prioritaskan diagonal jika dekat)
3:    $towerType \leftarrow DETECTPATTERNTYPEORALTERNATE(rc, ruin)$ 
4:   FILLTOWERPATTERNGREEDY( $rc, ruin, towerType$ )
5:   if  $CANCOMPLETETOWERPATTERN(rc, towerType, ruin)$  then
6:     COMPLETETOWERPATTERN( $rc, towerType, ruin$ )
7:   end if
8: end procedure
```



Algorithm 9 Fill Tower Pattern Greedy

```
1: procedure FILLTOWERPATTERNGREEDY(rc, ruin, towerType)
2:   for all tile pola  $5 \times 5$  di sekitar ruin do
3:     if tile valid (bukan wall, bukan center ruin) then
4:       desired  $\leftarrow$  warna primer/sekunder sesuai template towerType
5:       if paint(tile)  $\neq$  desired and CANATTACK(rc, tile) then
6:         ATTACK(rc, tile, desiredSecondary)
7:          $\triangleright$  Greedy: koreksi tile yang terlihat dan bisa diserang sekarang
8:       end if
9:     end if
10:  end for
11: end procedure
```

Algorithm 10 Explore Turn Soldier

```
1: procedure RUNEXPLORETURN(rc)
2:   FLAGENEMYPAINTNEARRUINS(rc)
3:    $\triangleright$  Soldier menaruh mark secondary agar mopper membersihkan area itu
4:   PAINTNEARBYUNCLAIMEDTILES(rc)
5:   MOVETOWARDTARGETUSINGBUG2(rc, target)
6:   if mencapai edge map then
7:     CHANGEDEFAULTTARGET(rc)
8:   end if
9: end procedure
```

Algorithm 11 Mopper Greedy

```
1: procedure RUNMOPPERGREEDY(rc)
2:   HANDLEMESSENGERMovement(rc)
3:   if MOVETOWARDFLAGGEDENEMY(rc) then
4:     return
5:   end if
6:   dir  $\leftarrow$  arah acak
7:   if CANMOVE(rc, dir) then
8:     MOVE(rc, dir)
9:   end if
10:  if CANMOPSWING(rc, dir) then
11:    MOPSWING(rc, dir)
12:  else if FRONTTILEHASENEMYPAINT(rc, dir) then
13:    ATTACKFRONTTILE(rc, dir)
14:  end if
15:  UPDATEENEMYROBOTSPeriodicReport(rc)
16: end procedure
```

Algorithm 12 Move Toward Flagged Enemy

```
1: function MOVETOWARDFLAGGEDENEMY(rc)
2:   bestFlag  $\leftarrow$  mark ally-secondary terdekat
3:   if bestFlag =  $\emptyset$  then
4:     return false
5:   end if
6:   for all tile sekitar bestFlag do
7:     if tile enemy paint and attackable then
8:       ATTACK(rc, tile)
9:       break
10:    end if
11:  end for
12:  Bergerak ke bestFlag
13:  if tidak ada enemy paint lagi di sekitar bestFlag then
14:    REMOVEMARK(rc, bestFlag)
15:  end if
16:  return true
17: end function
```

Algorithm 13 Tower Greedy (Saving-Aware Spawning)

```
1: procedure RUNTOWERGREEDY(rc)
2:   READMESSAGES(rc)
3:   if menerima pesan SAVING then
4:     skipTurns  $\leftarrow$  max(skipTurns, SAVE_SKIP_TURNS)
5:   end if
6:   if skipTurns > 0 then
7:     skipTurns  $\leftarrow$  skipTurns - 1
8:     return
9:   end if
10:  toBuild  $\leftarrow$  CHOOSEUNITTYPEBYPHASEANDMAPSCALE(rc)
11:  needed  $\leftarrow$  moneyCost(toBuild) + CHIP_RESERVE
12:  if chips < needed then
13:    return
14:                                      $\triangleright$  Hold chip agar completion tower tidak gagal
15:  end if
16:  spawnLoc  $\leftarrow$  FINDBESTSPAWNTILEGREEDY(rc, toBuild)
17:  spawnLoc  $\neq \emptyset$ 
18:  BUILDROBOT(rc, toBuild, spawnLoc)
19:
20:  if SHOULDUPGRADETOWER(rc) then
21:    UPGRADETOWER(rc)
22:  end if
23: end procedure
```

4.2.3 Struktur Data yang Digunakan

1. **allyTowersLoc** : **ArrayList<MapLocation>**
Menyimpan lokasi tower ally yang terlihat; dipakai messenger saat mode saving.
2. **reportedRuins** : **ArrayList<MapLocation>**
Menyimpan ruin yang sudah dilaporkan agar tidak duplikasi trigger saving.
3. **defaultLoc**, **target**, **spawnTower**, **closestRefill** : **MapLocation**
defaultLoc/*target* untuk eksplorasi, *spawnTower* untuk fallback pulang, *closestRefill* untuk isi ulang paint.
4. **isRefilling**, **isSaving**, **isMessenger** : **boolean**
Flag mode perilaku Soldier/Mopper.
5. **skipTurns** : **int**
Counter pause spawning pada tower saat ada pesan SAVING.

6. **Threshold paint: PAINT_LOW, PAINT_HIGH**
Membuat histeresis refill agar tidak toggle terus setiap turn.
7. **State bug2: isTracing, tracingDir, prevDest, line, obstacleStartDist**
Menyimpan konteks tracing rintangan agar pathfinding stabil.

4.2.4 Fungsi dan Prosedur Kunci

1. **bug2**: navigasi utama Thee, reset tracing jika target berubah.
2. **FindClosestEmptyRuinNotOccupied**: pilih ruin kosong terdekat yang belum diambil soldier lain.
3. **HandleRuinBuilding**: gerak-ke-ruin, isi pola, lalu complete tower.
4. **DetectPatternTypeOrAlternate**: lanjutkan pola existing atau fallback pola alternasi paint/-money.
5. **FlagEnemyPaintNearRuins**: Soldier menandai area yang butuh pembersihan.
6. **MoveTowardFlaggedEnemy**: Mopper mengejar mark dan membersihkan enemy paint.
7. **UpdateEnemyRobotsPeriodicReport**: pelaporan periodik jumlah musuh di sekitar.
8. **ChooseUnitTypeByPhaseAndMapScale**: komposisi spawn adaptif terhadap fase permainan.
9. **FindBestSpawnTileGreedy**: pilih tile spawn yang minim crowding dan aman.
10. **ShouldUpgradeTower**: upgrade konservatif, ditunda saat mode saving aktif.

4.2.5 Catatan Implementasi

Pada folder bot Thee saat ini, unit **SPLASHER** belum diimplementasikan (di dispatcher utama tidak ada pemanggilan perilaku splasher khusus). Karena itu, strategi greedy Thee berfokus pada kombinasi **SOLDIER + MOPPER + TOWER**.

4.3 Chamalet

4.3.1 Solusi Greedy yang Dipilih

Bot Chamalet menggunakan pendekatan **greedy modular** dengan komponen:

1. **Tower greedy economy**: spawn berdasarkan fase game dan buffer chips.
2. **Soldier greedy objective**: refill → bangun ruin → micro tower musuh → eksplorasi + paint.
3. **Mopper greedy control**: mop area ruin, mop enemy paint, combat, transfer paint.
4. **Splasher greedy pressure**: splash berdasarkan skor area dan kiting terhadap tower.
5. **Pathfinding bug2**: greedy move dengan fallback tracing dinding.

4.3.2 Pseudocode Detail

Algorithm 14 Main Loop Chamalet

```
1: procedure RUNUNITTURN(rc)
2:   if rc.type = SOLDIER then
3:     RUNSOLDIERGREEDY(rc)
4:   else if rc.type = SPLASHER then
5:     RUNSPLASHERGREEDY(rc)
6:   else if rc.type = MOPPER then
7:     RUNMOPPERGREEDY(rc)
8:   else
9:     RUNTOWERGREEDY(rc)
10:  end if
11: end procedure
```



Tower

Algorithm 15 Tower Greedy: Read Message, Attack, Spawn

```
1: procedure RUNTOWERGREEDY( $rc$ )
2:   for all msg in messages do
3:     if  $msg = SAVE\_CHIPS$  then
4:        $saveChipsUntil \leftarrow currentRound + 80$ 
5:     end if
6:   end for
7:   TOWERATTACKLOWESTHP( $rc$ )
8:   TOWERSPAWNBYPHASEANDBUFFER( $rc$ )
9: end procedure
```

Algorithm 16 Tower Attack Lowest HP

```
1: procedure TOWERATTACKLOWESTHP( $rc$ )
2:   pilih target musuh dalam range dengan:
     prioritas-1: HP paling rendah
     prioritas-2: jarak lebih dekat (tie-break)
3:   if target ada dan action ready then
4:     attack(target)
5:   end if
6: end procedure
```

Algorithm 17 Tower Spawn (Greedy Economy)

```
1: procedure TOWERSPAWNBYPHASEANDBUFFER( $rc$ )
2:   if tidak action ready then return
3:   end if
4:    $chipFloor \leftarrow 200$  jika early, selain itu 400
5:   if  $currentRound < saveChipsUntil$  then
6:      $chipFloor \leftarrow \max(chipFloor, 1100)$ 
7:   end if
8:   if  $chips < chipFloor$  then return
9:   end if
10:  pilih tipe unit:
     early: S, S, M (mod 3)
     mid/late: S, S, Sp, Sp, M, M, M (mod 7)
11:  cari tile spawn legal dengan skor minimum:
      $score = 10 \times \text{jumlah ally sekitar} + \text{penalti jika tile bukan ally paint}$ 
12:  if bestSpawn ditemukan then
13:    buildRobot(unitType, bestSpawn)
14:  end if
15: end procedure
```



Soldier

Algorithm 18 Soldier Greedy Objective Flow

```
1: procedure RUNSOLDIERGREEDY( $rc$ )
2:   REMEMBERNEARESTPAINTTOWER( $rc$ )
3:   if  $paint < SOLDIER\_PAINT\_THRESHOLD$  then
4:     if TRYWITHDRAWPAINT( $rc$ ) = false then
5:       SEEKREFUEL TOWERWITHBUG2( $rc$ )
6:     end if
7:     return
8:   end if
9:   if  $targetRuin$  sudah bertower then
10:     $targetRuin \leftarrow \emptyset$ 
11:  end if
12:  if  $targetRuin = \emptyset$  then
13:     $targetRuin \leftarrow$  FINDNEARESTUNBUILTRUINGREEDY( $rc$ )
14:
15:    if  $targetRuin \neq \emptyset$  then
16:      TRYBUILDTOWERATRUI N( $rc, targetRuin$ )
17:      return
18:    end if
19:    if TRYMICROATTACKENEMYTOWER( $rc$ ) then
20:      return
21:    end if
22:    EXPLOREMOVE( $rc$ )
23:    PAINTCURRENTTILE( $rc$ )
24:  end procedure
```

Algorithm 19 Try Build Tower at Ruin

```
1: procedure TRYBUILDTOWERATRUIN( $rc, ruin$ )
2:   if bisa kirim pesan ke tower ally terdekat then
3:     sendMessage(SAVE_CHIPS)
4:   end if
5:   if jarak ke ruin > 8 then
6:     BUG2( $rc, ruin$ )
7:     return
8:   end if
9:   tentukan tipe tower greedy dari rasio tower ally:
     jika paint tower kurang dari sepertiga total  $\Rightarrow$  build paint tower
     selain itu  $\Rightarrow$  build money tower
10:  if bisa mark pattern then
11:    markTowerPattern(towerType, ruin)
12:  end if
13:  cari tile pattern terdekat yang belum sesuai mark
14:  if pattern mengandung enemy paint dan tidak ada tile aman untuk lanjut then
15:    blok ruin sementara (cooldown) dan batal target
16:    return
17:  end if
18:  if tileToPaint ada then
19:    if tidak dalam range attack then
20:      BUG2( $rc, tileToPaint$ )
21:    end if
22:    if canAttack(tileToPaint) then
23:      attack(tileToPaint, sesuai primer/sekunder mark)
24:    end if
25:  else
26:    if jarak ke ruin > 2 then
27:      BUG2( $rc, ruin$ )
28:    end if
29:  end if
30:  if canCompleteTowerPattern then
31:    completeTowerPattern
32:     $targetRuin \leftarrow \emptyset$ 
33:  end if
34: end procedure
```

Algorithm 20 Find Nearest Unbuilt Ruin (Scored Greedy)

```
1: function FINDNEARESTUNBUILTRUINGREEDY( $rc$ )
2:    $best \leftarrow \emptyset, bestScore \leftarrow +\infty$ 
3:   for all ruin terdeteksi do
4:     if ruin di cooldown block atau sudah ada robot then continue
5:     end if
6:      $score \leftarrow distSq(me, ruin)$ 
7:     if ada enemy paint di pattern ruin then  $score \leftarrow score + RUIN\_PAINT\_PENALTY$ 
8:     end if
9:     if ada tower ally dekat ruin then  $score \leftarrow score - RUIN\_ALLY\_BONUS$ 
10:    end if
11:    if  $score < bestScore$  then
12:      update best
13:    end if
14:  end for
15:  return best
16: end function
```

Splasher

Algorithm 21 Splasher Greedy

```
1: procedure RUNSPLASHERGREEDY(rc)
2:   REMEMBERNEARESTPAINTTOWER(rc)
3:   if paint <  $0.15 \times \text{capacity}$  then
4:     if TRYWITHDRAWPAINT(rc) = false then
5:       SEEKREFUEL TOWERWITHBUG2(rc)
6:     end if
7:     return
8:   end if
9:   splashed  $\leftarrow$  TRYSPASHENEMYPAINTGREEDY(rc)
10:  cari tower musuh terdekat
11:  if movement ready then
12:    if tower musuh dekat (jarak kecil) then
13:      kite menjauh dari tower
14:    else
15:      dir  $\leftarrow$  GREEDYSPLASHERDIRECTION(rc)
16:      dir legal move(dir)
17:    end if
18:  end if
19:
20:  if splashed = false then
21:    TRYSPASHENEMYPAINTGREEDY(rc)
22:  end if
23: end procedure
```

Algorithm 22 Try Splash Enemy Paint (Scored Area)

```
1: function TRYSPASHENEMYPAINTGREEDY(rc)
2:  lindungi area pattern ruin yang belum bertower (no-fire zone)
3:  bestTarget  $\leftarrow \emptyset$ , bestScore  $\leftarrow 0$ 
4:  for all tile kandidat dalam radius aksi do
5:    if tidak bisa attack atau tile kena protected ruin pattern then continue
6:    end if
7:    hitung skor area sekitar kandidat:
      +4 untuk setiap enemy paint
      -2 untuk setiap ally paint
8:    if score > bestScore then
9:      update best
10:   end if
11: end for
12: if bestTarget  $\neq \emptyset$  then
13:   attack(bestTarget); return true
14: end if
15: return false
16: end function
```



Mopper

Algorithm 23 Mopper Greedy Pipeline

```
1: procedure RUNMOPPERGREEDY(rc)
2:   REMEMBERNEARESTTOWER(rc)
3:   sense enemies, allies
4:   if paint < refuelThreshold and no enemies then
5:     if TRYWITHDRAWPAINT(rc) = false then
6:       SEEKREFUELTOWERWITHBUG2(rc)
7:       return
8:     end if
9:   end if
10:  if action ready then MOPRUINAREA(rc)
11:  end if
12:  if action ready then MOPNEARESTENEMYPAINT(rc)
13:  end if
14:  if action ready and enemies exist then MOPPERCOMBAT(rc, enemies)
15:  end if
16:  if action ready then MOPPERTTRANSFERPAINT(rc, allies)
17:  end if
18:  if movement ready then
19:    dir ← GREEDYMOPPERDIRECTION(rc)
20:    dir legal move(dir)
21:  else
22:    bug2 ke enemy terdekat (jika ada)
23:  end if
24:
25:    if action ready then MOPNEARESTENEMYPAINT(rc)
26:    end if
27:    if action ready and enemies after move exist then MOPPERCOMBAT(rc, enemiesAfterMove)
28:    end if
29:  end procedure
```

Algorithm 24 Mopper Combat

```
1: procedure MOPPERCOMBAT(rc, enemies)
2:   cari cardinal swing terbaik (hit count maksimum)
3:   if bestSwingHits ≥ 2 then
4:     mopSwing(bestSwing); return
5:   end if
6:   pilih target non-tower yang bisa diattack dengan paintAmount tertinggi
7:   if target ada then
8:     attack(target); return
9:   end if
10:  if bestSwingHits ≥ 1 then
11:    mopSwing(bestSwing)
12:  end if
13: end procedure
```

Algorithm 25 Greedy Mopper Direction

```
1: function GREEDYMOPPERDIRECTION(rc)
2:   untuk setiap arah legal, hitung:
        $score = 2 \times \text{jumlah enemy paint sekitar destinasi}$ 
        $+ 3 \times \text{jumlah enemy robots sekitar destinasi}$ 
3:   pilih arah dengan skor maksimum
4:   if tidak ada arah baik then
5:     return explore direction
6:   end if
7:   return best direction
8: end function
```

4.3.3 Struktur Data yang Digunakan

1. **targetRuin, blockedRuin, blockedRuinRound**
Mengelola target pembangunan ruin dan anti-loop ruin bermasalah (cooldown).
2. **nearestTower**
Referensi tower terdekat untuk refill/withdraw paint.
3. **symmetryTarget**
Target eksplorasi simetri (perkiraan sisi lawan) untuk dorongan map control.
4. **saveChipsUntil**
Batas waktu mode hemat chips yang dipicu pesan SAVE_CHIPS.
5. **spawnCount**
Counter rotasi komposisi spawn tower.
6. **exploreDir**
Arah eksplorasi persisten ketika target eksplisit tidak tersedia.
7. **Pathfinding bug2 state: b2Tracing, b2StartDist, b2Dir, b2Dest, b2Line**
Menyimpan status tracing dinding dan garis referensi (M-line).

4.3.4 Fungsi dan Prosedur Kunci

1. **towerSpawn**: spawn adaptif fase game + kontrol buffer chip.
2. **towerAttack**: prioritas finishing musuh HP rendah.
3. **tryBuildTower**: pipeline bangun ruin end-to-end (signal, mark, paint, complete).
4. **findNearestUnbuiltRuin**: scoring ruin dengan penalti enemy paint dan bonus tower ally.
5. **tryMicroAttackEnemyTower**: serang tower musuh dengan kiting sederhana.
6. **trySplashEnemyPaint**: pemilihan pusat splash berbasis skor area.
7. **mopperCombat**: swing multi-hit, fallback single-target highest paint.
8. **mopperTransferPaint**: support ally berpaint ratio rendah.
9. **greedyMopperDirection/greedySplasherDirection**: pemilihan arah berdasarkan skor lokal.
10. **bug2**: pathfinding utama ketika direct move tidak stabil.

4.3.5 Catatan Inti Heuristik Chamalet

1. Komunikasi sengaja minimal: fokus pada SAVE_CHIPS untuk sinkronisasi ekonomi tower.
2. Pathfinding konsisten memakai bug2 dengan reset saat target berubah.
3. Micro agresif tetapi sederhana: kiting tower, mop combat berbasis value, dan transfer paint.
4. Macro tower menjaga keseimbangan: pressure unit tetap jalan, namun tidak mengorbankan completion tower.

4.4 Hasil Pertandingan dan Analisis

4.4.1 Himothee vs Thee

Skor: 74 - 1



Gambar 4.4.1: Himothee vs Thee

Dominasi karena ekonomi berbasis SRPs yang mengonsumsi lawan. 1 loss karena map "mit" di mana pathfinding bot himothee kurang unggul.

4.4.2 Himothee vs Chamalet

Skor: 71 - 4

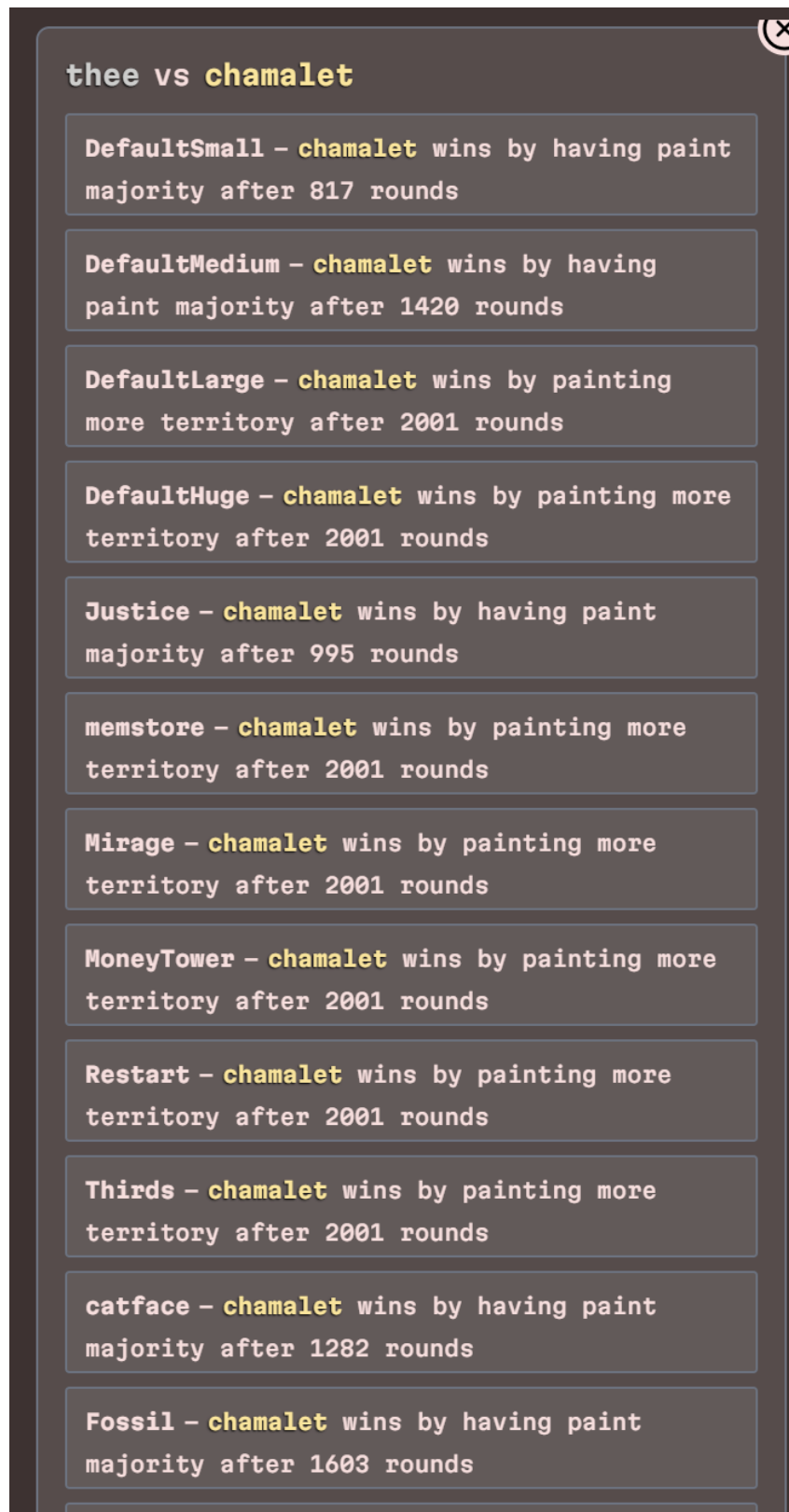


Gambar 4.4.2: Himothee vs Chamalet

Dominasi karena ekonomi juga. Losses terjadi mainly karena map-map di mana eksplorasi bebal bagi bot himothee.

4.4.3 Thee vs Chamalet

Skor: 6 - 69



Gambar 4.4.3: Thee vs Chamalet

Akar dari kekalahan Thee adalah deadweight Splasher yang tidak produktif. Losses Chamalet mainly datang dari eksplorasi early-game yang kurang dibandingkan dengan Thee

4.4.4 Local Postmortem Analysis

Himothée

- **Kelebihan:**

- Memiliki manajemen *pathfinding* yang efisien untuk mencapai *ruins* lebih cepat di awal permainan.
- Strategi *greedy* dalam penentuan lokasi *Special Resource Pattern* (SRP) cukup agresif, sehingga *ekonomi* cenderung melejit dibanding lawan.
- Penghematan paint dengan membangun towers tanpa melakukan marking. (namun pembangunan SRP masih membutuhkan marking).

- **Kekurangan:**

- Terlalu kaku dalam menyerang, seringkali *pathfinding* kurang optimal dan robot terjebak bergerak di tengah map, tanpa target yang pasti dan produktif.
- Manajemen *chips* terkadang kurang optimal saat ingin membangun tower baru, dan eksplorasi yang kurang komprehensif.
- Susah ketika berada di map yang *early-game exploration* tidak membuahkan banyak towers sehingga *ekonomi* tidak melonjak.

Thee

- **Kelebihan:**

- Arsitektur *delegation-based* yang jelas: *Soldier* menandai area bermasalah, *Mopper* mengeksekusi pembersihan. Hal ini menurunkan *overlap* tugas antar-unit.
- Efisien dalam pengelolaan *chips* melalui mekanisme *saving/skip turns*, sehingga peluang *tower completion* lebih stabil pada fase kritis.
- Implementasi *bug2* sederhana namun konsisten; saat target berubah, lintasan lama ditinggalkan sehingga respons terhadap objektif baru tetap cepat.

- **Kekurangan:**

- Ketergantungan tinggi pada *Mopper* sebagai *messenger*; jika unit ini tertekan, koordinasi *ekonomi* tim ikut melemah.
- *Splasher* belum menjadi komponen ofensif yang matang, sehingga tekanan area jarak menengah kurang optimal dibanding dua bot lain.
- Pendekatan makro relatif konservatif; pada kondisi tertentu dapat terlambat mengonversi momentum menjadi *map control* agresif.

Chamalet

- **Kelebihan:**

- *Micro* agresif dan efektif pada duel lokal: *kiting* terhadap menara lawan serta prioritas target HP rendah mempercepat *pick-off*.
- *Splasher* memiliki *scoring* area tembak yang baik, termasuk *blacklist* area pola sekutu agar tidak merusak progres pembangunan sendiri.
- Rotasi produksi unit sederhana namun adaptif, sehingga mudah menjaga tekanan tempur sambil mempertahankan cadangan *ekonomi* minimum.

- **Kekurangan:**

- Rentan terhadap *rush* ketika unit terlalu fokus membuka wilayah dan kurang disiplin menjaga formasi defensif individu.
- Dalam konfigurasi *walls* kompleks, *local search* pergerakan dapat menghasilkan rute tidak efisien atau gejala *looping*.
- Koordinasi komunikasi relatif minimalis (berat pada sinyal *save chips*), sehingga kualitas *global information sharing* masih di bawah Himothée.

5 Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian bot untuk persoalan Battlecode 2025, algoritma *greedy* berhasil diterapkan dengan baik pada tiga alternatif bot yang dibuat. Masing-masing bot menggunakan perhitungan heuristik yang berbeda untuk mengenali keadaan sekitar, memilih tindakan terbaik saat itu juga, dan menjalankannya sesuai target yang ditentukan. Dari ketiga pilihan tersebut, strategi pada bot utama akhirnya dipilih untuk digunakan karena terbukti paling efektif saat diadu satu lawan satu dengan dua bot alternatif lainnya. Bot utama ini menunjukkan hasil yang lebih konsisten selama simulasi pertandingan.

Dari pengujian yang dilakukan, terlihat jelas juga kelebihan dan kekurangan dari strategi *greedy* yang dipakai. Sisi positifnya, bot sangat cepat dalam pengambilan keputusan yang paling menguntungkan di depan mata. Namun, karena cara kerjanya yang tidak memikirkan rencana jangka panjang, bot kadang gagal mendapatkan hasil akhir yang maksimal. Keputusan bot sering kali menjadi kurang tepat ketika bertarung dalam peta yang rumit atau saat musuh bermain dengan strategi jangka panjang yang lebih kompleks.

5.2 Saran

Untuk pengembangan bot ke depannya, ada beberapa hal yang bisa diperbaiki supaya algoritma lebih maksimal. Pertama, perhitungan bobot-bobot variabel dalam strategi *greedy* bisa dibuat lebih dinamis menyesuaikan kondisi ronde. Contohnya, bot bisa diprogram supaya punya fokus yang berbeda antara awal permainan dengan akhir permainan, sehingga strateginya tidak terlalu kaku.

Selain masalah kode, proses pengujian bot juga sebaiknya diperluas dan tidak cuma berpusat pada pertandingan melawan bot buatan sendiri. Bot utama perlu banyak diuji dengan melawan berbagai macam bot dari orang lain yang punya strategi yang lebih sulit ditebak. Pengujian yang lebih luas ini sangat penting untuk mencari tahu celah kelemahan dari bot kita yang mungkin belum terlihat selama proses simulasi internal kemarin.



Lampiran

Repository Github : https://github.com/philipphqiwu/Tubes1_StimatheeChalamet

Video Penjelasan : <https://www.youtube.com/@vinr6074>

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Emilio Justin (13524043)

Philipp Hamara (13524101)

Vincent Rionarlie (13524031)



References

- MIT Battlecode (2025a), 'Battlecode 2025 game specifications', [Online]. Available: <https://releases.battlecode.org/specs/battlecode25/3.1.0/specs.pdf>. [Accessed: 27-Feb-2026].
- MIT Battlecode (2025b), 'Battlecode 2025 javadoc: Package battlecode.common', [Online]. Available: <https://releases.battlecode.org/javadoc/battlecode25/3.1.0/battlecode/common/package-summary.html>. [Accessed: 27-Feb-2026].
- Munir, R. (2026), 'Algoritma greedy (bagian 1)', [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-\(2026\)-Bag1.pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/04-Algoritma-Greedy-(2026)-Bag1.pdf). [Accessed: 12-Mar-2025].